



SIBDAS

Envio de Dados por POST

 **Inserir novo cliente**

Nome Completo

Pedro Santos

Morada (NºPorta, Andar)

Rua de Cima

Código Postal

4200-288

Cidade

Porto

Telefone

912345678

Email

snatos@gm,ail.com

Sexo

☐ Masculino ☐ Feminino

Data de nascimento

2025-05-13

Estado Civil

Escolha uma opção

Sistema de Saúde

Profissão

✕ Cancelar

💾 Guardar

Foram encontrados os seguintes erros:

- O endereço de email não é válido.
- O campo Género é obrigatório.
- Sistema de saúde não preenchido.
- Profissão é obrigatória.

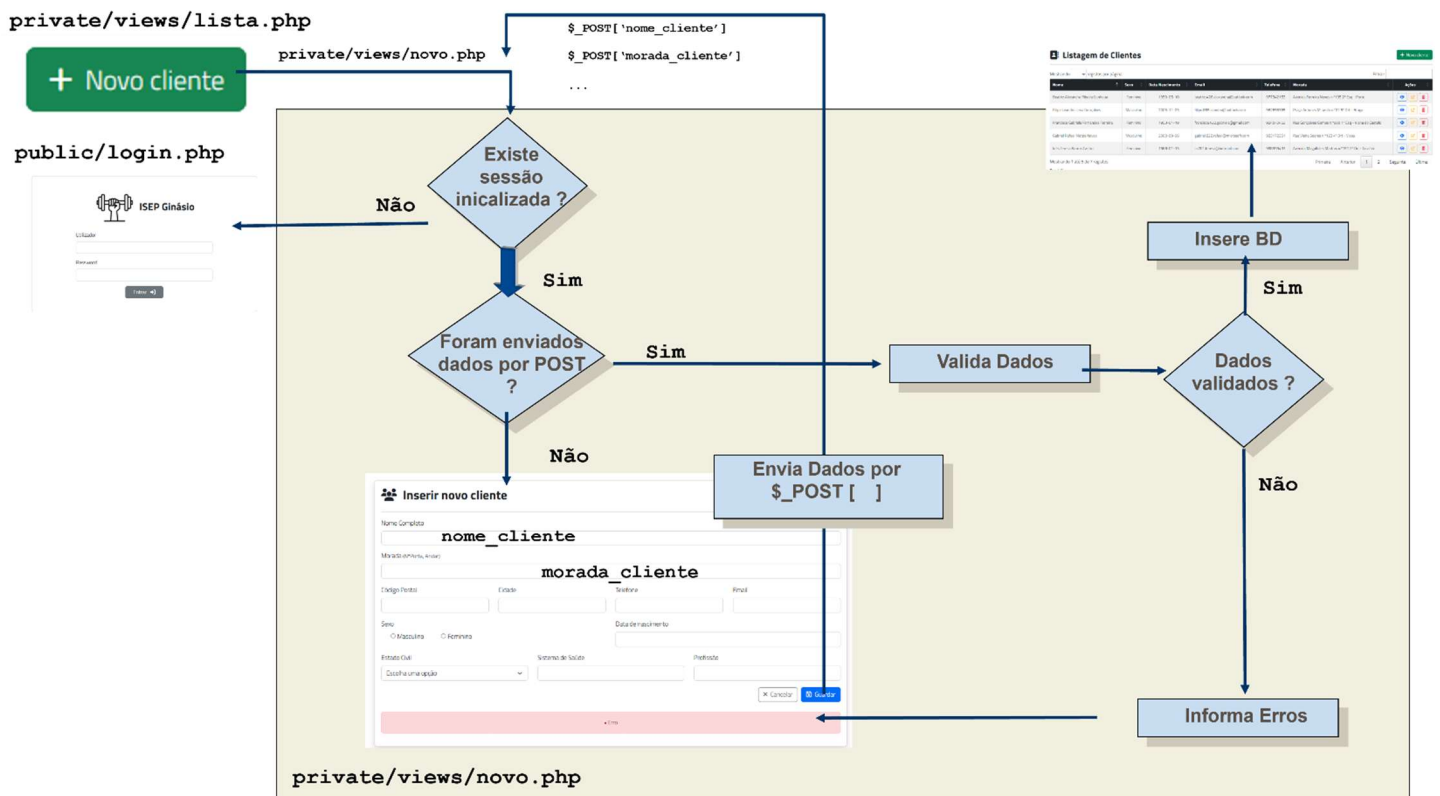
PHP e MySQL

1: Inserir Clientes (Método POST)

1 Inserir Clientes (Método POST)

Para se efetuar a **inserção de dados** no ficheiro novo.php, é necessário:

1. Recolher os dados do formulário
2. Validar os dados
3. Guardar os dados na base de dados
4. Exibir mensagem de sucesso ou erro
5. Manter os dados no formulário em caso de erro



Objetivo: Criar um formulário HTML básico com um campo para o nome

1. Abrir o ficheiro novo.php, que contém o formulário HTML e o código PHP
2. No formulário, identificar o campo correspondente ao nome completo e verificar qual o valor atribuído ao atributo name. Esse valor será utilizado posteriormente em PHP.

```
<form action="#" method="post" novalidate>
  <!-- Linhas e colunas com campos organizados -->
  <div class="row mb-3">
    <div class="col-12">
      <label for="texto_nome" class="form-label">Nome Completo</label>
      <input type="text" class="form-control" id="texto_nome" name="nome_cliente"
        required>
    </div>
  </div>
</div>
```

Nome Completo

nome_cliente

Atenção - Garantir o correto funcionamento do formulário

- O formulário deve conter o atributo `action="#"`, o que indica que os dados devem ser submetidos para a mesma página. Isso permite que o código PHP no topo do ficheiro trate os dados diretamente.
- O valor definido no atributo name (por exemplo, `"nome_cliente"`) corresponde exatamente ao nome que será utilizado para aceder ao valor submetido através do array `$_POST`, em PHP

Registar num caderno o nome desta variável (nome_cliente)

```
<input type="text" class="form-control" id="texto_nome" name="nome_cliente" required>
```

Objetivo: Garantir que o formulário submete corretamente os dados

3. Ir para o início (linha 1).do ficheiro novo.php.
4. Confirmar que o seguinte bloco de segurança (Sessão Iniciada) se encontra presente nas primeiras linhas do ficheiro PHP, antes de qualquer conteúdo HTML:

```
require_once __DIR__ . '/../../includes/funcoes.php';
redirect_if_not_logged();
```

Se o código não estiver presente, adicionar exatamente esse bloco no topo. Certificar de que está antes de qualquer outra linha de PHP ou HTML.

Objetivo: Processar os dados recebidos via POST e apresentar no ecrã

5. Verificar se o formulário foi submetido (via POST). Antes de recolher os dados enviados pelo utilizador, é necessário verificar se o formulário foi submetido. Isso é feito através da seguinte instrução

```
// Verificar se o formulário foi submetido
if ($_SERVER["REQUEST_METHOD"] == "POST") {

    // 1. Recolher dados
    $nome = $_POST["nome_cliente"] ?? "";

    // Imprimir os dados recebidos (para teste)
    echo "<p>Nome recebido: $nome</p>";

    // 2. Validar os dados
    // 3. Se não houver erros, guardar na base de dados
}
```

- Esta condição verifica **se a página recebeu dados através do método POST**, ou seja, se o formulário foi realmente enviado. Se a verificação for verdadeira, então os dados podem ser recolhidos e processados a seguir (validados, guardados, etc.).
- O operador ?? é o "null coalescing operator" em PHP. Retorna o valor à esquerda se este existir e não for null; caso contrário, devolve o valor à direita (valor por defeito).

Objetivo: Verificar o funcionamento do POST com um teste prático

6. Testar o envio de dados (funcionamento do POST). Para testar o formulário, deve-se introduzir um nome completo no campo correspondente e clicar no botão **Guardar**. O nome submetido deverá ser impresso na página como confirmação de que os dados foram enviados corretamente via método POST

Exemplo de comportamento esperado:

1. Preencher o campo "Nome Completo"
2. Clicar no botão "Guardar"
3. A página recarrega e imprime:
"Nome recebido: Pedro Sousa"

7. Confirmação do resultado Após submeter, será apresentada uma mensagem de teste com os dados recolhidos. Se o nome foi impresso corretamente, significa que o envio via POST funcionou.

Objetivo: Recolher e apresentar os dados de todos os campos do formulário e iniciar a validação dos dados inseridos.

1. Repetir para os restantes campos Substituir o bloco de recolha de apenas um campo por todos os campos do formulário.

```
// Verificar se o formulário foi submetido
if ($_SERVER["REQUEST_METHOD"] == "POST") {

    // 1. Recolher dados
    $nome      = $_POST["nome_cliente"]      ?? "";
    $morada     = $_POST["morada_cliente"]    ?? "";
    $cp         = $_POST["cp_cliente"]        ?? "";
    $cidade     = $_POST["cid_cliente"]       ?? "";
    $telefone   = $_POST["tel_cliente"]       ?? "";
    $email      = $_POST["email_cliente"]     ?? "";
    $sexo       = $_POST["radio_gender"]      ?? "";
    $dnasc      = $_POST["dnasc_cliente"]     ?? "";
    $estado     = $_POST["estativ_cliente"]   ?? "";
    $sistema    = $_POST["campo_opcao"]      ?? "";
    $profissao  = $_POST["profissao_cliente"] ?? "";

    // 2. Imprimir os dados recebidos (para teste)
    echo "<p><strong>Dados recebidos:</strong> Nome: $nome
| Morada: $morada | Código Postal: $cp | Cidade: $cidade
| Telefone: $telefone | Email: $email | Sexo: " . ($sexo == 'm' ? 'Masculino' : 'Feminino') . "
| Data de nascimento: $dnasc | Estado civil: $estado | Sistema de Saúde: $sistema
| Profissão: $profissao</p>";

    // 3. (Opcional) Validar os dados
    // 4. (Opcional) Guardar na base de dados
}
```

2. Preencher o formulário da seguinte forma


Inserir novo cliente

Nome Completo

Morada (NºPorta, Andar)

Código Postal

Cidade

Telefone

Email

Sexo

☒ Masculino
 ☐ Feminino

Data de nascimento

Estado Civil

Solteiro

Sistema de Saúde

Profissão

Cancelar

Guardar

Erro

Problemas com os dados enviados

Mesmo que os dados sejam enviados com sucesso (o que significa que o POST foi bem executado), **isso não significa que estão corretos ou válidos.**

Exemplo de problemas comuns:

- Nome com apenas letras maiúsculas ou apenas caracteres repetidos (AAA BBB CCC)
- Código postal incompleto
- Telefones com letras
- Email sem @

Nota: É necessário implementar **validações** que verifiquem:

- Campos obrigatórios
- Formatos corretos (ex: email, telefone)
- Conteúdos inválidos (ex: números em campos de texto)

Dados recebidos: Nome: AAAA | Morada: BBBB | Código Postal: CCCC | Cidade: DDDD | Telefone: EEEE | Email: FFF | Sexo: Masculino | Data de nascimento: GGGGG | Estado civil: solt | Sistema de Saúde: HHHH | Profissão: IIIII


ISEP Ginásio

Menu

Cientes

Agendamento de treinos

Planos de Treino

Equipamentos

Produtos e Serviços


Inserir novo cliente

Nome Completo

3. Validar o conteúdo do campo "Nome". Após recolher o valor do campo, é importante garantir:

- Que o campo não está vazio
- Que não contém **apenas números** ou **mistura de letras com números**

```
$nome = trim($nome);

// 1. Verificar se o campo está vazio
if (empty($nome)) {
    $erros[] = "O campo Nome é obrigatório.";
}

// 2. Verificar se contém apenas números ou mistura de letras com números
elseif (preg_match('/\d/', $nome)) {
    $erros[] = "O campo Nome não pode conter números.";
}
```

Explicação técnica:

- trim(\$nome) → remove espaços em branco do início e fim.
- preg_match('/\d/', \$nome) → procura se existe **algum dígito numérico (0-9)** no texto.
 - Apanha tanto nomes como "123" como "Maria123".

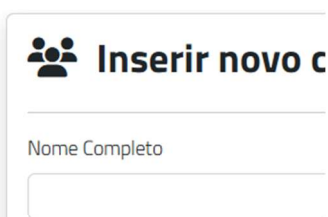
4. **Mostrar erros para depuração (fase de testes)** Para ver se os erros estão a ser corretamente armazenados:

```
// 4. Depuração: mostrar os erros recolhidos
echo "<pre>"; // torna mais legível no browser
print_r($erros);
echo "</pre>";
```

5 Testes

Caso 1: Campo vazio

- Resultado:



```
Array
(
    [0] => O campo Nome é obrigatório.
)
```

Caso 2: Letras e números misturados ou só números

Nome Completo

Dados recebidos: Nome: Pedro 2222 | Morada: | Código Postal: | C

```

Array
(
    [0] => O campo Nome não pode conter números.
)

```

Caso 3: Nome válido

Nome Completo

```

Array
(
)

```

Objetivo: Implementar validações para todos os campos do formulário e garantir que os dados introduzidos incorretamente são assinalados sem apagar os restantes dados válidos inseridos.

1. Evitar que os dados desapareçam em caso de erro. Por padrão, se houver erro e o formulário for recarregado, os dados preenchidos desaparecem. Para manter os valores preenchidos:

Substituir:

```
<input type="text" class="form-control" id="texto_nome" name="nome_cliente"
required>
```

Por

```
<input type="text" name="nome_cliente" class="form-control"
value="<?= $_POST['nome_cliente'] ?? ' ' ?>">
```

Confirmar que tudo funciona

O formulário agora:

- Valida os campos corretamente
- Informa o utilizador dos erros
- Mantém os dados preenchidos mesmo com erro

Resultado visível: o campo continua com o valor digitado (Pedro222) e a mensagem de erro aparece.

2024-2025

Dados recebidos: Nome: Pedro222 | Morada: | Código Postal: | Cidade: | Telefone: | Email: | S

```
Array
(
    [0] => 0 campo Nome não pode conter números.
)
```



2. Acrescentar as restantes validações

```
// 3. Validar os dados
$erros = [];
$nome = trim($nome);
$morada = trim($morada);
$cp = trim($cp);
$cidade = trim($cidade);
$telefone = trim($telefone);
$email = trim($email);
$estado = trim($estado);
$sistema = trim($sistema);
$profissao = trim($profissao);

// Verificar se o campo está vazio
if (empty($nome)) {
    $erros[] = "O campo Nome é obrigatório.";
}
// Verificar se contém apenas números ou mistura de letras com números
elseif (preg_match('/\d/', $nome)) {
    $erros[] = "O campo Nome não pode conter números.";
}

if (empty($morada)) $erros[] = "O campo Morada é obrigatório.";

// Verificar se o campo está vazio
if (empty($cp)) {
    $erros[] = "O campo Código Postal é obrigatório.";
}
// Verificar se o formato está correto (0000-000)
elseif (!preg_match('/^\d{4}-\d{3}$/ ', $cp)) {
    $erros[] = "Código Postal inválido (ex: 4000-007).";
}

if (empty($cidade)) $erros[] = "O campo Cidade é obrigatório.";

// Verificar se o campo está vazio
if (empty($telefone)) {
    $erros[] = "O campo Telefone é obrigatório.";
}
// Verificar se o número de telefone é válido (ex: 912345678)
elseif (!preg_match('/^9\d{8}$/ ', $telefone)) {
    $erros[] = "O número de telefone não é válido. Deve começar por 9 e ter 9 dígitos.";
}

// Verificar se o campo está vazio
if (empty($email)) {
    $erros[] = "O campo Email é obrigatório.";
}
// Verificar se o formato do email é válido
elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $erros[] = "O endereço de email não é válido.";
}

if (empty($_POST['radio_gender'])) {
    $erros[] = "O campo Género é obrigatório.";
}
```

```
if (empty($estado)) $erros[] = "Estado civil não selecionado.";
if (empty($sistema)) $erros[] = "Sistema de saúde não preenchido.";
if (empty($profissao)) $erros[] = "Profissão é obrigatória.";
```

Explicação técnica:

- `preg_match('/^\d{4}-\d{3}$/', $cp) → exige:`
 - exatamente **4 dígitos**
 - um **hífen**
 - exatamente **3 dígitos**
- O padrão original com `-?` aceitava código postal sem hífen.
Agora exige sempre o formato **com hífen**, como é correto em Portugal.
- `preg_match('/^9\d{8}$/', $telefone)`
 - `^9` → o número deve começar por 9
 - `\d{8}` → seguido de exatamente 8 dígitos
 - Total: 9 dígitos (ex: 912345678)

3 Testes

Caso 1: Campos vazios

Inserir novo cliente

Nome Completo

Morada (NºPorta, Andar)

Código Postal

Cidade

Sexo

☒ Masculino ☐ Feminino

Estado Civil

Sistema de Saúde

Array

(

```
[0] => O campo Nome é obrigatório.
[1] => O campo Morada é obrigatório.
[2] => O campo Código Postal é obrigatório.
[3] => O campo Cidade é obrigatório.
[4] => O campo Telefone é obrigatório.
[5] => O campo Email é obrigatório.
[6] => Sistema de saúde não preenchido.
[7] => Profissão é obrigatória.
```

)

Objetivo: Manter os valores preenchidos pelo utilizador no formulário mesmo após um erro de validação, garantindo uma melhor experiência de utilização e evitando perdas de dados.

1. Evitar que os dados desapareçam em caso de erro. Por padrão, se houver erro e o formulário for recarregado, os dados preenchidos desaparecem. Para manter os valores preenchidos dos restantes campos:

- **Morada**

Substituir:

```
<input type="text" class="form-control" id="texto_endereco"
      name="morada_cliente">
```

Por

```
<input type="text" class="form-control" id="texto_endereco" name="morada_cliente"
      value="<?>= htmlspecialchars($_POST['morada_cliente']) ?>">
```

- **Código postal**

Substituir

```
<input type="text" class="form-control" id="texto_cp" name="cp_cliente" required>
```

por

```
<input type="text" name="cp_cliente" class="form-control"
      value="<?>= htmlspecialchars($_POST['cp_cliente']) ?>">
```

- **Cidade**

Substituir:

```
<input type="text" class="form-control" id="texto_cidade" name="cid_cliente"
      required>
```

Por

```
<input type="text" class="form-control" id="texto_cidade" name="cid_cliente"
      value="<?>= htmlspecialchars($_POST['cid_cliente']) ?>" required>
```

- **Telefone**

Substituir

```
<input type="text" class="form-control" id="texto_cliente" name="tel_cliente" required>
```

por

```
<input type="text" class="form-control" id="texto_cliente" name="tel_cliente"
      value="<?>= htmlspecialchars($_POST['tel_cliente']) ?>" required>
```

- **Email**

Substituir

```
<input type="email" class="form-control" id="texto_email" name="email_cliente" required>
```

por

```
<input type="email" name="email_cliente" class="form-control"
value="<?=$_POST['email_cliente'] ?? '' ?>">
```

- **Sexo**

Substituir

```
<input class="form-check-input" type="radio" name="radio_gender"
id="radio_m" value="m" checked>
<input class="form-check-input" type="radio" name="radio_gender"
id="radio_f" value="f">
```

por

```
<input type="radio" name="radio_gender" value="m"
<?=(($_POST['radio_gender'] ?? '') === 'm') ? 'checked' : '' ?>>

. . .
<input type="radio" name="radio_gender" value="f"
<?=(($_POST['radio_gender'] ?? '') === 'f') ? 'checked' : '' ?>>
```

- **Estado Civil**

Substituir

```
<option value="solt">Solteiro</option>
<option value="casd">Casado</option>
<option value="ufat">União de Facto</option>
```

por

```
<option value="solt" <?=(($_POST['estaciv_cliente'] ?? '') == 'solt') ? 'selected' : '' ?>>Solteiro</option>
<option value="casd" <?=(($_POST['estaciv_cliente'] ?? '') == 'casd') ? 'selected' : '' ?>>Casado</option>
<option value="ufat" <?=(($_POST['estaciv_cliente'] ?? '') == 'ufat') ? 'selected' : '' ?>>União de Facto</option>
```

▪ Sistema de Saude

Substituir

```
<input type="text" class="form-control" id="texto_SSaude" name="campo_opcao"
      list="sistemasasaude">
```

por

```
<input type="text" class="form-control" id="texto_SSaude" name="campo_opcao"
      list="sistemasasaude" value="<?=<= htmlspecialchars($_POST['campo_opcao']) ?> ">
```

▪ Profissão

Substituir

```
<input type="text" class="form-control" id="profissao" name="profissao_cliente">
```

por

```
<input type="text" class="form-control" id="profissao" name="profissao_cliente"
      value="<?=<= htmlspecialchars($_POST['profissao_cliente']) ?> ">
```

Objetivo: Garantir que os dados recolhidos do formulário são armazenados de forma consistente e padronizada, independentemente da forma como o utilizador os introduziu.

1. Após validar os dados (isto é, garantir que estão preenchidos e no formato correto), deve ser feita a **normalização** dos mesmos. Isso assegura:

- Capitalização adequada nos nomes
- Formatação uniforme de cidade e profissão
- E-mails sempre em minúsculas
- Sistemas de saúde em letras maiúsculas

Código de normalização de entrada

```
// Normalizar entrada
$nome      = ucwords(strtolower($nome));           // Pedro Santos
$cidade    = ucfirst(strtolower($cidade));         // Porto
$email     = strtolower($email);                   // guimas@email.pt
$sistema   = strtoupper($sistema);                 // ADSE
$profissao = ucwords(strtolower($profissao));       // Professor
```

2024-2025

Variável	Transformação aplicada	Exemplo antes → depois
\$nome	strtolower + ucwords → tudo minúsculo, depois capitalizar cada palavra	"mARIA sOUsa" → "Maria Sousa"
\$cidade	strtolower + ucfirst → primeira letra maiúscula apenas	"PORTO" → "Porto"
\$email	strtolower → todo em minúsculas	"ANA@GMAIL.COM" → "ana@gmail.com"
\$sistema	strtoupper → tudo em maiúsculas	"sns" → "SNS"
\$profissao	strtolower + ucwords → capitalizar palavras	"tecnico de informatica" → "Tecnico De Informatica"

2. Exemplo de teste prático. Após a normalização adicione este bloco para imprimir os dados normalizados:

```
echo "<p><strong>Dados normalizados:</strong></p>";
echo "<ul>";
echo "<li>Nome: $nome</li>";
echo "<li>Cidade: $cidade</li>";
echo "<li>Email: $email</li>";
echo "<li>Sistema de Saúde: $sistema</li>";
echo "<li>Profissão: $profissao</li>";
echo "</ul>";
```

3. No formulário, preencha os seguintes campos da seguinte forma:

- Nome: mARIA sOUsa
- Cidade: PORTO
- Email: ANA@GMAIL.COM
- Sistema: sns
- Profissão: tecnico de informatica

4. Após clicar em **Guardar**, deverá ver impresso no ecrã:

Dados recebidos: Nome: mARIA sOUsa | Mor
nformatica

Dados normalizados:

- Nome: Maria Sousa
- Cidade: Porto
- Email: ana@gmail.com
- Sistema de Saúde: SNS
- Profissão: Tecnico De Informatica

Este teste confirma que, **independentemente de como o utilizador escreve os dados**, o sistema assegura consistência e padronização antes de qualquer registo na base de dados.

5. Depois de confirmar que os dados estão a ser corretamente normalizados e apresentados, o código de teste deve ser **comentado** para que deixe de ser executado.

```
/*  
echo "<p><strong>Dados normalizados:</strong></p>";  
.  
.  
echo "</ul>";  
*/
```

Objetivo: Permitir que o campo de **data de nascimento** utilize um calendário gráfico moderno e uniforme, facilitando a seleção correta da data pelo utilizador..

1. Fazer o download do Flatpickr

- Aceder ao **Moodle da disciplina**, na **Ficha 12**.
- Fazer o **download da pasta flatpickr**, que contém:
 - flatpickr.min.css
 - flatpickr.js

2. Colocar a pasta no projeto

- Copiar a pasta flatpickr para dentro da pasta assets do projeto.
- Ficheiro final deverá estar em:
 - /assets/flatpickr/flatpickr.min.css
 - /assets/flatpickr/flatpickr.js

3. Incluir o Flatpickr no **header.php**. Abrir o ficheiro **includes/header.php** e adicionar as seguintes linhas, antes do encerramento da tag <head>:

```
<!-- CSS do Flatpickr -->  
<link rel="stylesheet" href="/isep-ginasio/private/assets/flatpickr/flatpickr.min.css">  
  
<!-- JS do Flatpickr -->  
<script src="/isep-ginasio/private/assets/flatpickr/flatpickr.js"></script>
```

4. Criar o campo "Data de Nascimento" no formulário

Substituir:

```
<input type="text" class="form-control" id="texto_dnasc" name="dnasc_cliente"
    required>
```

por

```
<input type="text" class="form-control" id="data_nasc" name="dnasc_cliente"
    value="<?= htmlspecialchars($_POST['dnasc_cliente'] ?? '') ?>" required>
```

5 . Ativar o calendário com flatpickr

Colocar este script no final da página

```
<script>
flatpickr("#data_nasc", {
    dateFormat: "Y-m-d"
});
</script>
```

6. Validar a data no PHP (backend)

Colocar o código seguinte na validação dos campos.

```
$dnasc = $_POST["dnasc_cliente"] ?? "";
$dnasc = trim($dnasc);

if (empty($dnasc)) {
    $erros[] = "O campo Data de Nascimento é obrigatório.";
}
// Validação de formato: AAAA-MM-DD
elseif (!preg_match('/^\d{4}-\d{2}-\d{2}$/', $dnasc)) {
    $erros[] = "Formato de data inválido. Use AAAA-MM-DD.";
}
// Verificar se é uma data real (ex: não aceitar 2024-02-31)
else {
    $partes = explode('-', $dnasc);
    if (!checkdate((int)$partes[1], (int)$partes[2], (int)$partes[0])) {
        $erros[] = "Data de nascimento inválida.";
    }
}
}
```

Valor introduzido	Resultado
(vazio)	Erro: campo obrigatório
2024-02-31	Erro: data inválida
1998-13-10	Erro: data inválida (mês 13 não existe)
1990-05-22	Válido

Passar os erros para baixo paa aparte vermelha e comentar o código recebido

Substituir

```
<!-- Área de erros -->
<div class="alert alert-danger text-center" role="alert">
  • Erro
</div>
```

Por

```
<!-- Área de erros -->
<?php if (!empty($erros)): ?>
  <div class="alert alert-danger" role="alert">
    <strong>Foram encontrados os seguintes erros:</strong>
    <ul class="mb-0">
      <?php foreach ($erros as $erro): ?>
        <li><?= htmlspecialchars($erro) ?></li>
      <?php endforeach; ?>
    </ul>
  </div>
<?php endif; ?>
```

 **Inserir novo cliente**

Nome Completo

Morada (NºPorta, Andar)

Código Postal

Cidade

Telefone

Email

Sexo
☐ Masculino ☐ Feminino

Data de nascimento

Estado Civil

Sistema de Saúde

Profissão

Foram encontrados os seguintes erros:

- O endereço de email não é válido.
- O campo Género é obrigatório.
- Sistema de saúde não preenchido.
- Profissão é obrigatória.

Comentar para não aparecer no topo

```
/*  
echo "<pre>"; // torna mais legível no browser  
print_r($erros);  
echo "</pre>";  
*/
```

```
/*  
echo "<p><strong>Dados recebidos:</strong> Nome: $nome  
| Morada: $morada | Código Postal: $cp | Cidade: $cidade  
| Telefone: $telefone | Email: $email | Sexo: " . ($sexo == 'm' ? 'Masculino' : 'Feminino') . "  
| Data de nascimento: $dnasc | Estado civil: $estado | Sistema de Saúde: $sistema  
| Profissão: $profissao</p>";  
*/
```

Objetivo: Se não existirem erros de validação, os dados normalizados e verificados devem ser inseridos na base de dados utilizando PDO

1. Preparar a ligação e executar INSERT com interpolação direta

```
if (empty($erros)) {  
    try {  
        $ligacao = new PDO(  
            "mysql:host=" . MYSQL_HOST . ";dbname=" . MYSQL_DATABASE . ";charset=utf8",  
            MYSQL_USERNAME,  
            MYSQL_PASSWORD  
        );  
  
        $ligacao->exec("INSERT INTO clientes (  
            nome, sexo, data_nascimento, email, telefone, morada, cidade, cliente_ativo, sistema_saude  
        ) VALUES (  
            '$nome', '$sexo', '$dnasc', '$email', '$telefone', '$morada', '$cidade', '1', '$sistema'  
        )");  
  
        header('Location: lista.php');  
        exit;  
    } catch (PDOException $err) {  
        $erros = "Erro ao gravar os dados: " . $err->getMessage();  
    }  
  
    $ligacao = null;  
}
```

Nesta fase, estamos a utilizar um INSERT direto com variáveis incorporadas na string SQL.

Este método **é funcional**, mas contém riscos:

- Pode causar erros se houver caracteres especiais (ex: aspas simples ').
- Pode abrir portas a injeções SQL.

Vamos verificar o que acontece se algo correr mal.

2. Testes de Validação

- Tentar submeter o formulário com dados válidos. Se os dados estiverem corretos vai para a lista de clientes. Verifique se o cliente que inseriu já se encontra na lista.

- No campo cidade por exemplo coloque ' ou ' ; -- . O resultado será um **erro MySQL**, mas este **não aparecerá na lista de erros de validação**, porque não é detetado por validações em PHP.

3. Preparar variáveis para distinguir tipos de erro. Antes da validação geral, criar uma variável adicional

```
// 3. Validar os dados
$erros = []; // para erros de validação
$erro_sistema = ""; // Para erros de SQL (PDO)
```

4. Capturar o erro do PDO (no catch)

```
$erro_sistema = "Erro ao gravar os dados: " . $err->getMessage();
```

5. Mostrar erro de sistema no HTML. Inserir logo após a área de apresentação de erros de validação:

```
<?php if (!empty($erro_sistema)): ?>
    <div class="alert alert-danger">
        <strong>Erro:</strong>
        <p><?= htmlspecialchars($erro_sistema) ?></p>
    </div>
<?php endif; ?>
```

5. Testar novamente com ' ; -- no campo cidade

Resultado esperado:

- O formulário falha ao inserir.
- É apresentado o **erro real de sintaxe do MySQL** no ecrã, por exemplo:

Erro:
Erro ao gravar os dados: SQLSTATE[42000]: Syntax error or access violation: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '--; '1'; 'ASDAS' ')' at line 4

Objetivo: Substituir a execução direta do SQL com variáveis interpoladas por uma abordagem segura com `prepare()` e `execute()` usando parâmetros nomeados.

Preparar a query com parâmetros nomeados

1. Criar a query com placeholders. Substitui os valores diretos como '\$nome' por **placeholders nomeados**, como :nome, :sexo, etc.

```
$sql = "INSERT INTO clientes (  
nome, sexo, data_nascimento, email, telefone, morada, cidade, cliente_ativo, sistema_saude  
) VALUES ( :nome, :sexo, :dnasc, :email, :telefone, :morada, :cidade, '1', :sistema)";
```

2. Preparar a query com prepare()

Ao invés de executar diretamente a SQL com `exec()`, usamos `prepare()`:

```
$stmt = $ligacao->prepare($sql);
```

3. Passar os dados com execute()

Aqui passamos um array associativo onde cada **chave** é um placeholder, e o **valor** é a variável PHP correspondente:

```
$stmt->execute([  
    ':nome' => $nome,  
    ':sexo' => $sexo,  
    ':dnasc' => $dnasc,  
    ':email' => $email,  
    ':telefone' => $telefone,  
    ':morada' => $morada,  
    ':cidade' => $cidade,  
    ':sistema' => $sistema  
]);
```

5. Código final no contexto do bloco try

```
if (empty($erros)) {  
    try {  
        $ligacao = new PDO(  
            "mysql:host=" . MYSQL_HOST . ";dbname=" . MYSQL_DATABASE . ";charset=utf8",  
            MYSQL_USERNAME,  
            MYSQL_PASSWORD  
        );  
  
        $sql = "INSERT INTO clientes (  
            nome, sexo, data_nascimento, email, telefone,  
            morada, cidade, cliente_ativo, sistema_saude  
        ) VALUES (  
            :nome, :sexo, :dnasc, :email, :telefone,  
            :morada, :cidade, '1', :sistema  
        )";  
  
        $stmt = $ligacao->prepare($sql);  
        $stmt->execute([  
            ':nome'      => $nome,  
            ':sexo'      => $sexo,  
            ':dnasc'     => $dnasc,  
            ':email'     => $email,  
            ':telefone'  => $telefone,  
            ':morada'    => $morada,  
            ':cidade'    => $cidade,  
            ':sistema'   => $sistema  
        ]);  
  
        header("Location: lista.php");  
        exit;  
    } catch (PDOException $err) {  
        $erro_sistema = "Erro ao gravar os dados: " . $err->getMessage();  
    }  
  
    $ligacao = null;  
}
```

6 Testes práticos

Teste 1 — Inserção normal (dados válidos)

- Preenche o formulário com dados reais (sem erros).
- Verifica se o cliente aparece na base de dados.
- Resultado: Inserção bem-sucedida.

Teste 2 — Tentativa de injeção SQL

Inserir no campo “Cidade”: Lisboa’); DROP TABLE clientes; --

- Com prepare() ativo, esta string será **guardada como texto**, não executada.
- Resultado: Dados inseridos com segurança. Nenhuma tabela é afetada.